Pertemuan 7 – Struktur Lebih Rapi (Refactoring)

1. Tujuan Refactoring (Kenapa perlu dipisah)
   a. sebelumnya (1 file) dan semua jadi satu di main.py. Dimana logic API, Model dan routing dalam 1 file, sehingga sulit dibaca dan dikembangkan
   b. Sesudah (dipisah) user_service/ menjadi beberapa file yaitu main.py digunakan sebagai entry point, file models.py sebagai struktur data dan file routes.py digunakan sebagai endpoint API. sehingga tampilan lebih rapi, scalable, dan profesional.
2. Contoh implementasi untuk folder user_service
   a. models.py

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
```

   b. main.py

```
from fastapi import FastAPI
from .routes import router

app = FastAPI()
app.include_router(router)
```

   c. routers.py

```
from fastapi import APIRouter, HTTPException
from .models import User

router = APIRouter()

users = []

@router.post("/users")
def create_user(user: User):
    if not user.name:
        raise HTTPException(status_code=400, detail="Nama tidak boleh
kosong")

    new_user = {"id": len(users)+1, "name": user.name}
    users.append(new_user)
    return new_user


@router.get("/users")
def get_users():
```

```
    return users
```

untuk menjalankannya perintahnya juga sama yaitu *"uvicorn user_service.main:app --port 8001 --reload"*

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> uvicorn user_service.main:app --po
rt 8001 --reload
INFO:     Will watch for changes in these directories: ['D:\\DosenQ\\Mengajar\\20252\\Sistem T
erdistribusi\\Praktikum']
INFO:     Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO:     Started reloader process [19016] using StatReload
INFO:     Started server process [27700]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

3. Contoh implementasi Product Service
   a. models.py

```
from pydantic import BaseModel

class Product(BaseModel):
    name: str
```

   b. main.py

```
from fastapi import FastAPI
from .routes import router  # pastikan file routes.py

app = FastAPI()

app.include_router(router)

# endpoint test
@app.get("/")
def home():
    return {"message": "API aktif"}
```

   c. routers.py

```
from fastapi import APIRouter, HTTPException
from .models import Product

router = APIRouter()
products = []

@router.post("/products")
def create_product(product: Product):
    # validasi sederhana
    if not product.name:
```

```
        raise HTTPException(status_code=400, detail="Nama produk tidak
boleh kosong")

    new_product = {
        "id": len(products)+1,
        "name": product.name
    }

    products.append(new_product)
    return new_product

@router.get("/products")
def get_products():
    return products
```

Untuk menjalankannya perintahnya juga sama yaitu *"uvicorn product_service.main:app --port 8002 --reload"*

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> uvicorn product_service.main:app -
-port 8002 --reload
INFO:     Will watch for changes in these directories: ['D:\\DosenQ\\Mengajar\\20252\\Sistem T
erdistribusi\\Praktikum']
INFO:     Uvicorn running on http://127.0.0.1:8002 (Press CTRL+C to quit)
INFO:     Started reloader process [28252] using StatReload
INFO:     Started server process [6424]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

4. Contoh implementasi Order Service
   a. models.py

```
from pydantic import BaseModel

class Order(BaseModel):
    user_id: int
    product_id: int
```

   b. main.py

```
from fastapi import FastAPI
from .routes import router

app = FastAPI()
app.include_router(router)
```

   c. routers.py

```
from fastapi import APIRouter, HTTPException
import requests
from .models import Order

router = APIRouter()
```

```
orders = []

@router.post("/orders")
def create_order(order: Order):
    # Ambil user dan product dari service lain
    try:
        users = requests.get("http://localhost:8001/users",
timeout=5).json()
        products = requests.get("http://localhost:8002/products",
timeout=5).json()
    except:
        raise HTTPException(status_code=500, detail="Service tidak
tersedia")

    selected_user = next((u for u in users if u["id"] == order.user_id),
None)
    selected_product = next((p for p in products if p["id"] ==
order.product_id), None)

    if not selected_user or not selected_product:
        raise HTTPException(status_code=404, detail="User atau Product
tidak ditemukan")

    new_order = {
        "id": len(orders)+1,
        "user": selected_user,
        "product": selected_product
    }
    orders.append(new_order)
    return new_order

@router.get("/orders")
def get_orders():
    return orders
```

Untuk menjalankannya perintahnya juga sama yaitu *"uvicorn order_service.main:app --port 8003 --reload"*

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> uvicorn order_service.main:app --p
ort 8003 --reload
INFO:     Will watch for changes in these directories: ['D:\\DosenQ\\Mengajar\\20252\\Sistem T
erdistribusi\\Praktikum']
INFO:     Uvicorn running on http://127.0.0.1:8003 (Press CTRL+C to quit)
INFO:     Started reloader process [9224] using StatReload
INFO:     Started server process [31360]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

5. Contoh implementasi Gateway
   a. main.py

```
from fastapi import FastAPI
```

```
from .routes import router

app = FastAPI()
app.include_router(router)
```

b. models.py

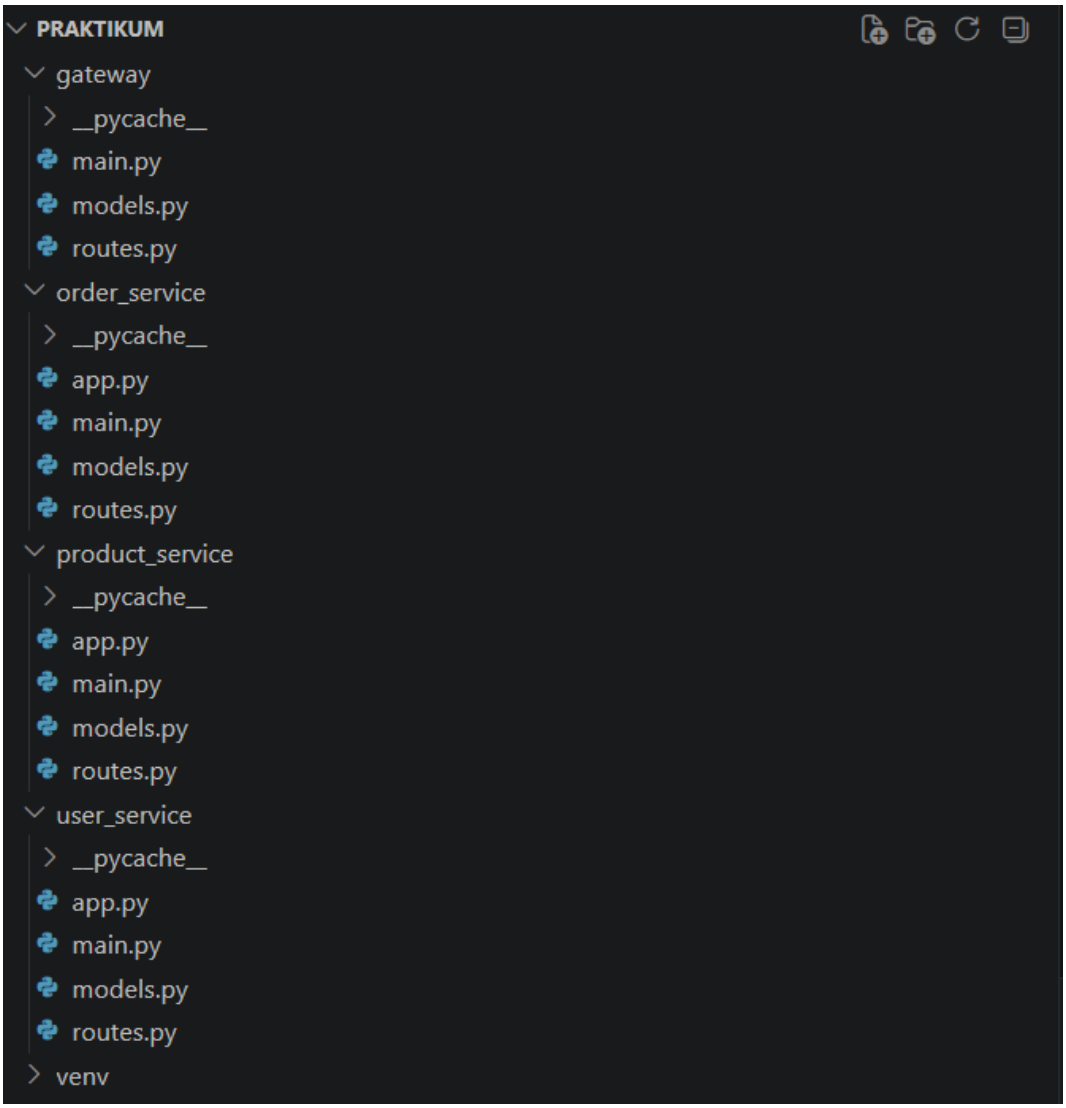
```
from pydantic import BaseModel

class Order(BaseModel):
    user_id: int
    product_id: int
```

c. routers.py

```
from fastapi import APIRouter
import requests

router = APIRouter()

# ====================
# USER SERVICE
# ====================

@router.get("/users")
def get_users():
    return requests.get("http://localhost:8001/users").json()

@router.post("/users")
def create_user(user: dict):
    return requests.post("http://localhost:8001/users", json=user).json()


# ====================
# PRODUCT SERVICE
# ====================

@router.get("/products")
def get_products():
    return requests.get("http://localhost:8002/products").json()

@router.post("/products")
def create_product(product: dict):

    # Simpan hasil request ke variable response
    response = requests.post(
        "http://localhost:8002/products",
        json=product
```

```
    )

    # Return hasil dari product service
    return {
        "status_code": response.status_code,
        "content": response.json()  # gunakan json() bukan text
    }


# ====================
# ORDER SERVICE
# ====================

@router.get("/orders")
def get_orders():
    return requests.get("http://localhost:8003/orders").json()

@router.post("/orders")
def create_order(order: dict):
    return requests.post("http://localhost:8003/orders",
json=order).json()
```

Untuk menjalankannya perintahnya juga sama yaitu ”*uvicorn gateway.main:app --port 8000 --reload*”

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> uvicorn gateway.main:app --port 80
00 --reload
INFO:     Will watch for changes in these directories: ['D:\\DosenQ\\Mengajar\\20252\\Sistem T
erdistribusi\\Praktikum']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [16832] using StatReload
INFO:     Started server process [19908]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

6. Sehingga struktur akhirnya akan seperti gambar dibawah berikut ini.